Name: Saai Vardhan D
Registration number : 192525205

CSA 0421 - OPERATING SYSTEM

Experiment 31

Construct a C program to simulate the First in First Out paging technique of memory management.

Executable C code :

```
#include <stdio.h>

int main() {
    int n, f, pages[30], frame[10], i, j, k = 0, faults = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter the reference string: ");
    for (i = 0; i < n; i++) scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);

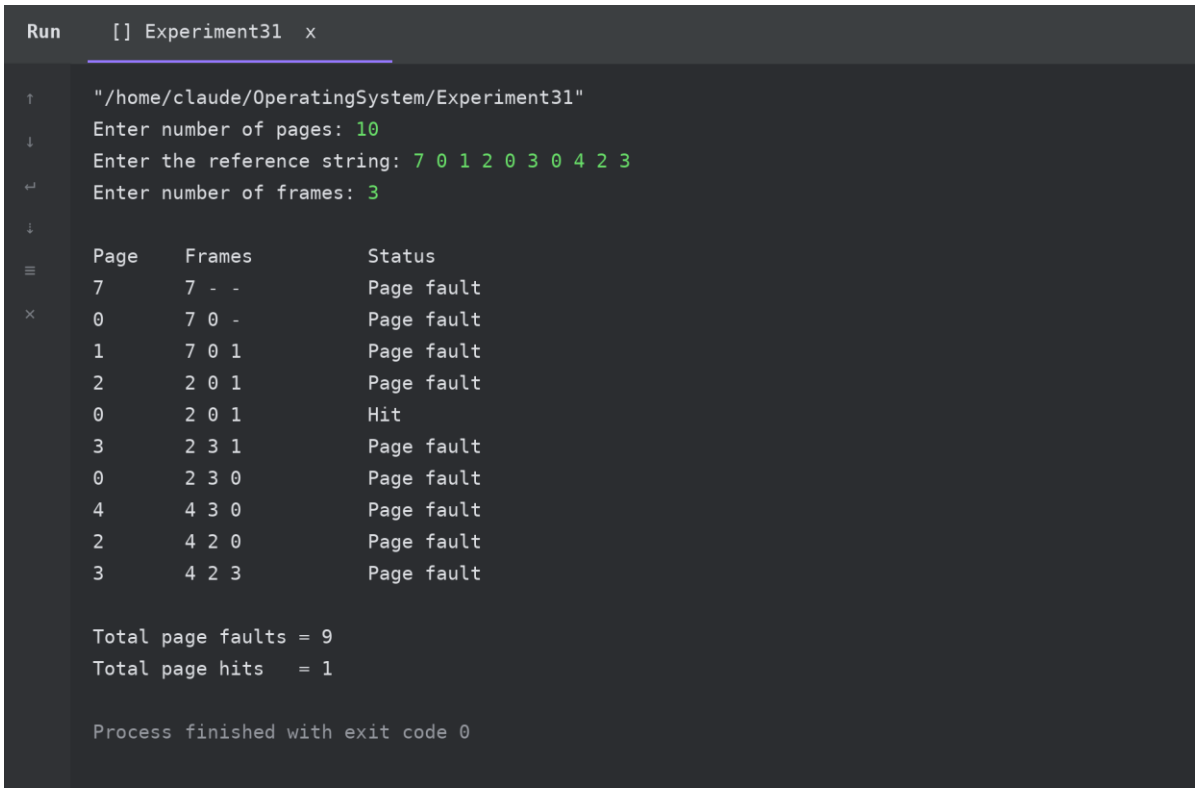
    for (j = 0; j < f; j++) frame[j] = -1;

    printf("\nPage\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < f; j++)
            if (frame[j] == pages[i]) found = 1;

        if (!found) {                    // page fault: replace the oldest page
            frame[k] = pages[i];
            k = (k + 1) % f;
            faults++;
        }

        printf("%d\t", pages[i]);
        for (j = 0; j < f; j++) {
            if (frame[j] == -1) printf("- ");
            else printf("%d ", frame[j]);
        }
        printf("\t\t%s\n", found ? "Hit" : "Page fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    printf("Total page hits   = %d\n", n - faults);
    return 0;
}
```

Output Screenshot:



Experiment 32

Construct a C program to simulate the Least Recently Used paging technique of memory management.

Executable C code :

```
#include <stdio.h>

int main() {
    int n, f, pages[30], frame[10], last[10], i, j, faults = 0, found, pos,
victim;

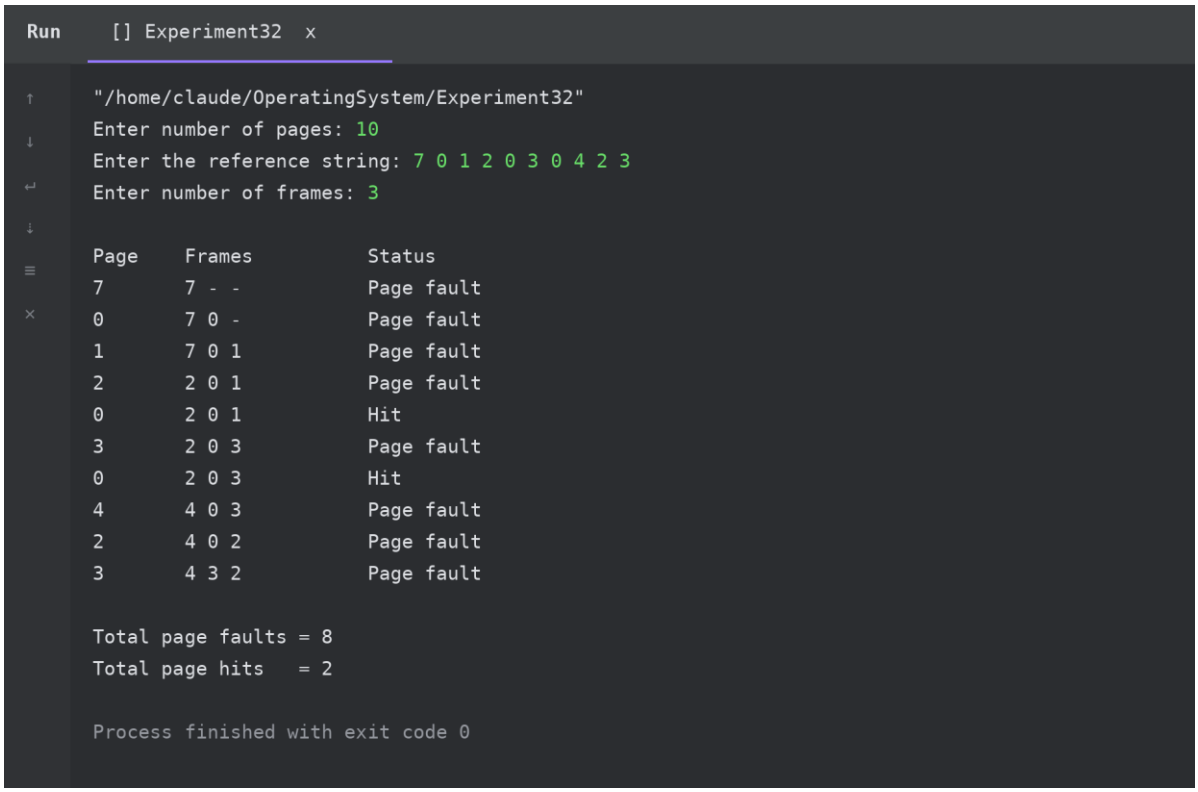
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter the reference string: ");
    for (i = 0; i < n; i++) scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);

    for (j = 0; j < f; j++) { frame[j] = -1; last[j] = -1; }

    printf("\nPage\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < f; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                last[j] = i;             // remember when it was used
                break;
            }
        }
        if (!found) {
            pos = -1;
            for (j = 0; j < f; j++)
                if (frame[j] == -1) { pos = j; break; }
            if (pos == -1) {             // no empty frame: replace the least
recently used page
                victim = 0;
                for (j = 1; j < f; j++)
                    if (last[j] < last[victim]) victim = j;
                pos = victim;
            }
            frame[pos] = pages[i];
            last[pos] = i;
            faults++;
        }

        printf("%d\t", pages[i]);
        for (j = 0; j < f; j++) {
            if (frame[j] == -1) printf("- ");
            else printf("%d ", frame[j]);
        }
        printf("\t\t%s\n", found ? "Hit" : "Page fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    printf("Total page hits   = %d\n", n - faults);
    return 0;
}
```

Output Screenshot:



Experiment 33

Construct a C program to simulate the optimal paging technique of memory management

Executable C code :

```
#include <stdio.h>

int main() {
    int n, f, pages[30], frame[10], i, j, k, faults = 0, found, pos, victim,
next, farthest;

    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter the reference string: ");
    for (i = 0; i < n; i++) scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);

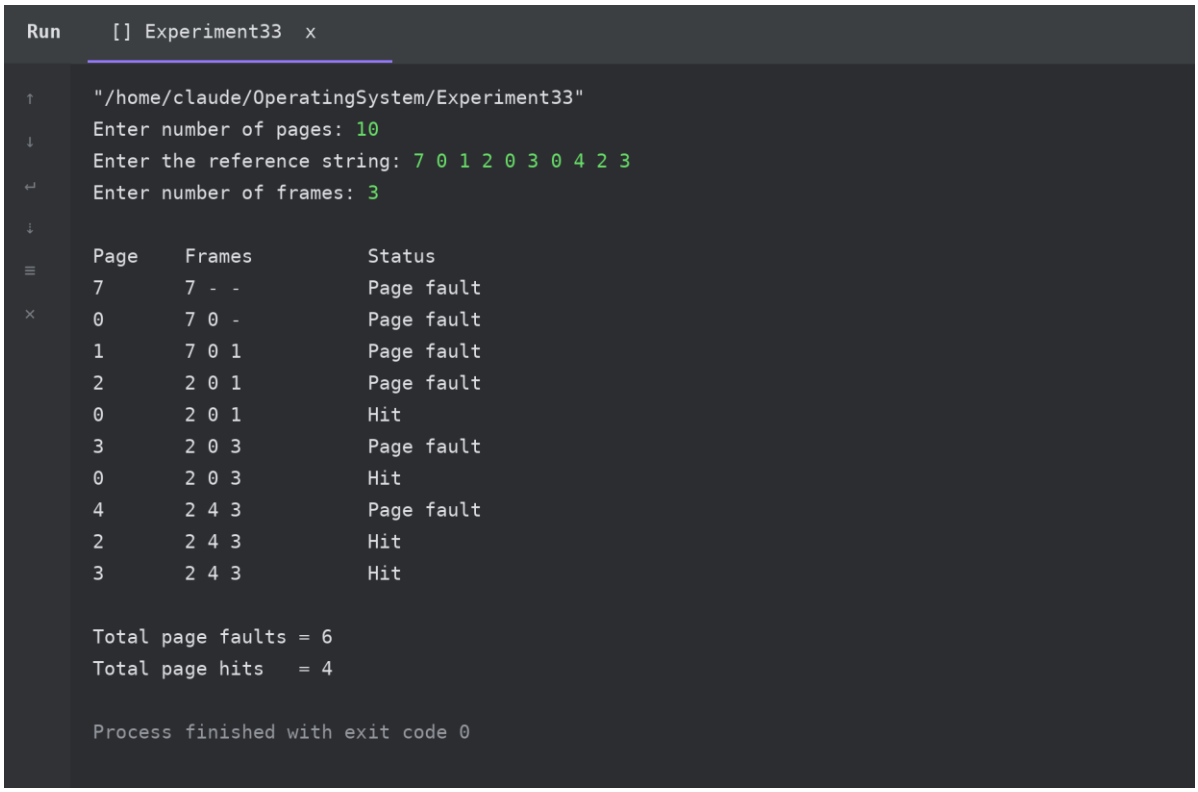
    for (j = 0; j < f; j++) frame[j] = -1;

    printf("\nPage\tFrames\t\tStatus\n");
    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < f; j++)
            if (frame[j] == pages[i]) found = 1;

        if (!found) {
            pos = -1;
            for (j = 0; j < f; j++)
                if (frame[j] == -1) { pos = j; break; }
            if (pos == -1) {              // replace the page that will be used
farthest in the future
                farthest = -1;
                victim = 0;
                for (j = 0; j < f; j++) {
                    next = n;             // n means "never used again"
                    for (k = i + 1; k < n; k++)
                        if (pages[k] == frame[j]) { next = k; break; }
                    if (next > farthest) { farthest = next; victim = j; }
                }
                pos = victim;
            }
            frame[pos] = pages[i];
            faults++;
        }

        printf("%d\t", pages[i]);
        for (j = 0; j < f; j++) {
            if (frame[j] == -1) printf("- ");
            else printf("%d ", frame[j]);
        }
        printf("\t\t%s\n", found ? "Hit" : "Page fault");
    }
    printf("\nTotal page faults = %d\n", faults);
    printf("Total page hits   = %d\n", n - faults);
    return 0;
}
```

Output Screenshot:



Experiment 34

Consider a file system where the records of the file are stored one after another both physically and logically. A record of the file can only be accessed by reading all the previous records. Design a C program to simulate the file allocation strategy.

Executable C code :

```
#include <stdio.h>

int main() {
    int blocks[20] = {0}, n, i, j, start, len, ok, clash;

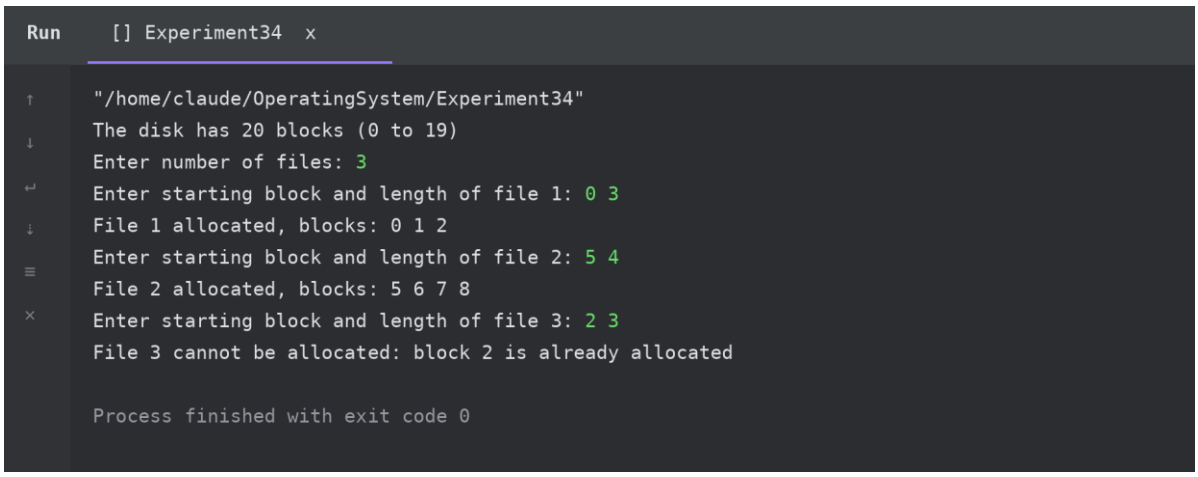
    printf("The disk has 20 blocks (0 to 19)\n");
    printf("Enter number of files: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        printf("Enter starting block and length of file %d: ", i);
        scanf("%d %d", &start, &len);

        ok = 1;
        clash = -1;
        if (start < 0 || start + len > 20) {
            ok = 0;
        } else {
            for (j = 0; j < len; j++) {
                if (blocks[start + j]) { ok = 0; clash = start + j; break; }
            }
        }

        if (ok) {
            printf("File %d allocated, blocks:", i);
            for (j = 0; j < len; j++) {
                blocks[start + j] = 1;
                printf(" %d", start + j);
            }
            printf("\n");
        } else if (clash != -1) {
            printf("File %d cannot be allocated: block %d is already
allocated\n", i, clash);
        } else {
            printf("File %d cannot be allocated: blocks go beyond the disk\n",
i);
        }
    }
    return 0;
}
```

Output Screenshot:



Experiment 35

Consider a file system that brings all the file pointers together into an index block. The ith entry in the index block points to the ith block of the file. Design a C program to simulate the file allocation strategy.

Executable C code :

```
#include <stdio.h>

int main() {
    int used[30] = {0}, n, i, j, idx, cnt, b[10], ok;

    printf("The disk has 30 blocks (0 to 29)\n");
    printf("Enter number of files: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        printf("\nEnter index block of file %d: ", i);
        scanf("%d", &idx);
        printf("Enter number of blocks needed: ");
        scanf("%d", &cnt);
        printf("Enter the block numbers: ");
        for (j = 0; j < cnt; j++) scanf("%d", &b[j]);

        ok = 1;
        if (idx < 0 || idx > 29 || used[idx]) {
            printf("File %d cannot be allocated: index block %d is not free\n",
i, idx);
            continue;
        }
        for (j = 0; j < cnt; j++) {
            if (b[j] < 0 || b[j] > 29 || used[b[j]] || b[j] == idx) {
                printf("File %d cannot be allocated: block %d is not free\n", i,
b[j]);
                ok = 0;
                break;
            }
        }
        if (ok) {
            used[idx] = 1;
            printf("File %d allocated. Index block = %d\n", i, idx);
            for (j = 0; j < cnt; j++) {
                used[b[j]] = 1;
                printf("  Index[%d] -> Block %d\n", j, b[j]);
            }
        }
    }
    return 0;
}
```

Output Screenshot:

